Module III

Bottom Up Parsing

Bottom up Parsing

2 Types

- Operator precedence parser
- LR Parsers

Bottom up Parsing

- Bottom up parsing is a process reducing a string w to start symbol of the grammar.

- At each reduction step, a specific substring matching the body of the production is replaced by the non terminal at the head of that production.

E → E + T | T

T →T * F | F

F → (E)| id

Construct “id * id”

Bottom up Parsing

- The key decision during bottom up parsing are about
  - When to reduce
  - What production to apply

- Right most derivation in reverse order can be obtained by handle pruning.

- Handle – non terminal

Handle

- A handle of a string is a substring that matches the right side of a production and whose reduction to the non terminal on the left side of the production represents one step along the reverse of a right most derivation.

Shift Reduce Parsing

- Shift Reduce Parser|Compiler Design|Part# 15|Learn CSE Malayalam

Shift Reduce Parsing

- A general form of bottom up syntax analysis is called shift reduce parsing.
- Generally used by LR parsers.
- Operator precedence parser is also a form of shift reduce parsing.

Shift Reduce Parsing

1. **Input Buffer**: This stores the string or sequence of tokens that needs to be parsed.

2. **Stack**: The parser uses a stack to keep track of which symbols or parts of the parse it has already processed. As it processes the input, symbols are pushed onto and popped off the stack.

3. **Parsing Table**: Similar to a predictive parser, a parsing table helps the parser decide what action to take next.

Shift Reduce Parsing – Before Parsing

- A $ is placed at the stack top

- A $ is append to the right end of input w$

Shift Reduce Parsing – After Parsing

- The stack contains $ along with the start symbol
- The input buffer contains the symbol $

Shift Reduce Parsing – Basic Operations

1. **Shift :** Shift symbols from input buffer onto the stack
2. **Reduce :** If the handle appears on top of the stack, then its reduction by using appropriate production rule is done.
3. **Accept :** Successful parsing is done
4. **Error:** The parser can neither perform shift action nor reduce action, and not even accept action.

Shift Reduce Parsing – Example

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

Construct “id * id”

Shift Reduce Parsing – Parsing Table

Stack	Input Buffer	Action
$	id*id $	Shift
$id	*id$	Reduce F → id
$F	*id$	Reduce T → F
$T	*id$	Shift
$T*	id$	Shift
$T*id	$	Reduce F → id
$T*F	$	Reduce T → T * F
$T	$	Reduce E → T
$E	$	Accept

Conflicts

- Every shift reduce parser can reach a configuration in which a parser knowing the entire stack contents and the next input symbol.
- Can’t decide whether to shift or to reduce called as shift reduce conflicts
- Can’t decide which of the several reductions to be made called as the reduce reduce conflicts.

Advantages

- Shift-reduce parsing is efficient and can handle a wide range of context-free grammars.
- It can parse a large variety of programming languages and is widely used in practice.
- It is capable of handling both left- and right-recursive grammars, which can be important in parsing certain programming languages.
- The parse table generated for shift-reduce parsing is typically small, which makes the parser efficient in terms of memory usage.

Disadvantages

- Shift-reduce parsing has a limited lookahead, which means that it may miss some syntax errors that require a larger lookahead.
- It may also generate false-positive shift-reduce conflicts, which can require additional manual intervention to resolve.
- Shift-reduce parsers may have difficulty in parsing ambiguous grammars, where there are multiple possible parse trees for a given input sequence.
- In some cases, the parse tree generated by shift-reduce parsing may be more complex than other parsing techniques.

Example

- Consider the grammar

  S –> (L) | a

  L –> L , S | S

  Perform Shift Reduce parsing for input string “(a, (a, a)) “.

Example

Stack	Input Buffer	Parsing Action
$	(a , (a , a)) $	Shift
$ (	a , (a , a)) $	Shift
$ (a	, (a , a)) $	Reduce S → a
$ (S	, (a , a)) $	Reduce L → S
$ (L	, (a , a)) $	Shift
$ (L ,	(a , a)) $	Shift
$ (L , (	a , a)) $	Shift
$ (L , (a	, a)) $	Reduce S → a
$ (L , (S	, a)) $	Reduce L → S
$ (L , (L	, a)) $	Shift
$ (L , (L ,	a)) $	Shift
$ (L , (L , a	)) $	Reduce S → a
$ (L, (L, S	)) $	Reduce L →L, S
$ (L, (L	)) $	Shift
$ (L, (L)	) $	Reduce S → (L)
$ (L, S	) $	Reduce L → L, S
$ (L	) $	Shift
$ (L)	$	Reduce S → (L)
$ S	$	Accept

University Questions

1. What are the possible action of a shift reduce parser? (3 marks)
2. What are viable prefixes? (3 marks)
3. What is shift reduce parser? Explain in detail the conflicts that may occur during shift reduce parsing? (8 marks)
4. What are handle pruning? Indicate the handles in the reduction of the sentence aaabbb to the start symbol using the grammar

S → aABb

A → aA | a

B → bB | b (8 marks)

Operator Precedence Parser

- Operator Precedence Parser|Compiler Design|Part# 16|Learn CSE Malayalam

Operator Precedence Parser

- Operator precedence parser is a bottom up parser, that interprets an operator precedence grammar.

- An operator precedence grammar is used to define mathematical operators.

- Calculator is using operator precedence parser.

Operator Precedence Parser

- Operator precedence parser is the simple shift reduce parser, that is capable of parsing a subset of LR(1) grammar.

- Ambiguous grammar are not allowed in any parsers except operator precedence parsers.

Operator Precedence Parser

- In an operator precedence grammar

1. There exits no production rules which contain ε on its RHS.
2. There exists no production rules which contains two non terminals adjacent to each other in RHS.

Eg:

E → EAE | id

A → + | -

E → E + E | E – E | id

Operator Precedence Parser

Three operator precedence relations are

a) If terminal “a” has highest precedence than terminal “b”

$$a \gtrdot b$$

b) If terminal “a” has lower precedence than terminal “b”

$$a \lessdot b$$

c) If both terminals “a” and “b” have same precedence

$$a \doteq b$$

Operator Precedence Parser

- Other precedence rules are:

1. An identifier is always given the highest precedence than any other symbols.
2. $ symbol is always given the lowest precedence.
3. If two operators have the same precedence, then go by checking their associativity.

Operator Precedence Parser - Example

- Operator Precedence Parser Example|Compiler Design|Part# 17|Learn CSE Malayalam

Operator Precedence Parser - Example

E → E + E | E * E | id

Operator precedence Table

	id	+	*	$
id	-	$\dot{>}$	$\dot{>}$	$\dot{>}$
+	$\dot{<}$	$\dot{>}$	$\dot{<}$	$\dot{>}$
*	$\dot{<}$	$\dot{>}$	$\dot{>}$	$\dot{>}$
$	$\dot{<}$	$\dot{<}$	$\dot{<}$	-

Operator Precedence parsing algorithm

- Input : An input string w and a table of precedence relations.
- Output : If w forms a skeletal parse tree with a placeholder non-terminal E labeling all interior nodes, it is accepted; otherwise, an error is indicated
- Method : Initially the stack contains $ and input string w$

Operator Precedence parsing algorithm

Steps:

1) Set ip to point to the first symbol of w$;
2) repeat forever
3) If $ is on top of the stack and ip points to $ then
4) return
5) else begin
6) Let ‘a’ be the topmost terminal symbol on the stack and let ‘b’ be the symbol pointing by ip;
7) If $a \lessdot b$ or $a \doteq b$ then begin

Operator Precedence parsing algorithm

8) push ‘b’ onto the stack;
9) Advance ip to the next input symbol;
10) end;
11) else if a $\gtrdot$ b then
12) Repeat
13) Pop the stack
14) Until the top stack terminal is related by to the terminal most recently popped.
15) else error
16) end

Operator Precedence Parser - Example

Operator Precedence Parser Example|Compiler Design|Part# 17|Learn CSE Malayalam - YouTube

Operator Precedence Parser - Example

$E \rightarrow E + E \mid E * E \mid id$

id + id * id

Operator Precedence Parser - Example

Stack	Input	Action
$	id+id*id$	Shift($<id)
$id	+id*id$	Reduce E→ id (id > +)
$	+id*id$	Shift ($<+)
$+	id*id$	Shift(+ < id)
$ + id	* id $	Reduce E→ id (id > *)
$ +	* id $	Shift (+ < *)
$ + *	id $	Shift (*<id)
$ + * id	$	Reduce E→ id (id > $)
$ + *	$	Reduce E → E * E (* > $)
$ +	$	Reduce E → E + E (+ > $)
$	$	Accept

Error identification

1. If no precedence relation holds between the terminals on the top of the stack and the current input.
2. If a handle has been found, but there is no production with this handle in right hand side.

Operator Precedence Parser

- Operator Precedence Parser Advantages and Disadvantages|Compiler Design|Part# 18|Learn CSE Malayalam

Advantages

1. Simple and easy to construct
2. Powerful enough for expressions in programming language
3. Syntactic errors like missing operand or operators, unbalanced right parenthesis or missing right parenthesis etc. can be easily identified.

Disadvantages

- It can't handle tokens like unary operators (eg. Unary minus)
  - The lexical analyzer should handle the unary minus
  - The lexical analyzer will return two different tokens for the unary minus and binary minus
- The handling of tokens known to have two different precedence become difficult
- Only a small class of grammars can be parsed using operator precedence parser

Disadvantages

- Since the relationship between a grammar for the language being parsed and the operator precedence parser itself is tenuous, one can’t always be sure the parser accepts exactly the desired language – week relationship between language and operator precedence parser

University Question

1. Illustrate actions according to operator precedence parser for the input id1+id2*id3 based on the given grammar.

$$E \rightarrow E+E \mid E*E \mid (E) \mid id$$

2. Explain how to construct an operator grammar without ambiguity.

Operator function Table

- Operator Function Table|Compiler Design|Part# 19|Learn CSE Malayalam

Operator function Table

- The major disadvantage of operator precedence table is that if we have n operators then the size of the table will be n*n and the complexity will be $O(n^2)$.

- In order to decrease the size of this table the operator function table is used.

Operator function Table

- Usually the operator precedence parsers do not store the precedence table with the relation.

- The table can be encoded by two precedence functions ‘f’ and ‘g’ that map terminal symbols to integers.

Operator function Table

- The precedence relations between the symbols are implemented by numerical comparison.

- For symbols ‘a’ and ‘b’
  - $f(a) < g(b)$ if $a \lessdot b$
  - $f(a) > g(b)$ if $a \gtrdot b$
  - $f(a) = g(b)$ if $a \doteq b$

Operator function Table - Algorithm

1. Create functions f(a) for each terminal ‘a’ and for the end of string symbol($)
2. Partition the created symbols into groups in such a way that if a $\doteq$ b , then f(a) and g(b) are in the same group.
3. Create a directed graph whose nodes are in the groups.
   1. For any a and b, if a $\lessdot$ b , place an edge from group of g(b) to the group of f(a)
   2. If a $\gtrdot$ b, place an edge from the group of f(a) to g(b).

Operator function Table - Algorithm

4. If the constructed graph has a cycle then no precedence functions exist. When there are no cycles, collect the length of the longest path from the groups of f(a) and g(b).

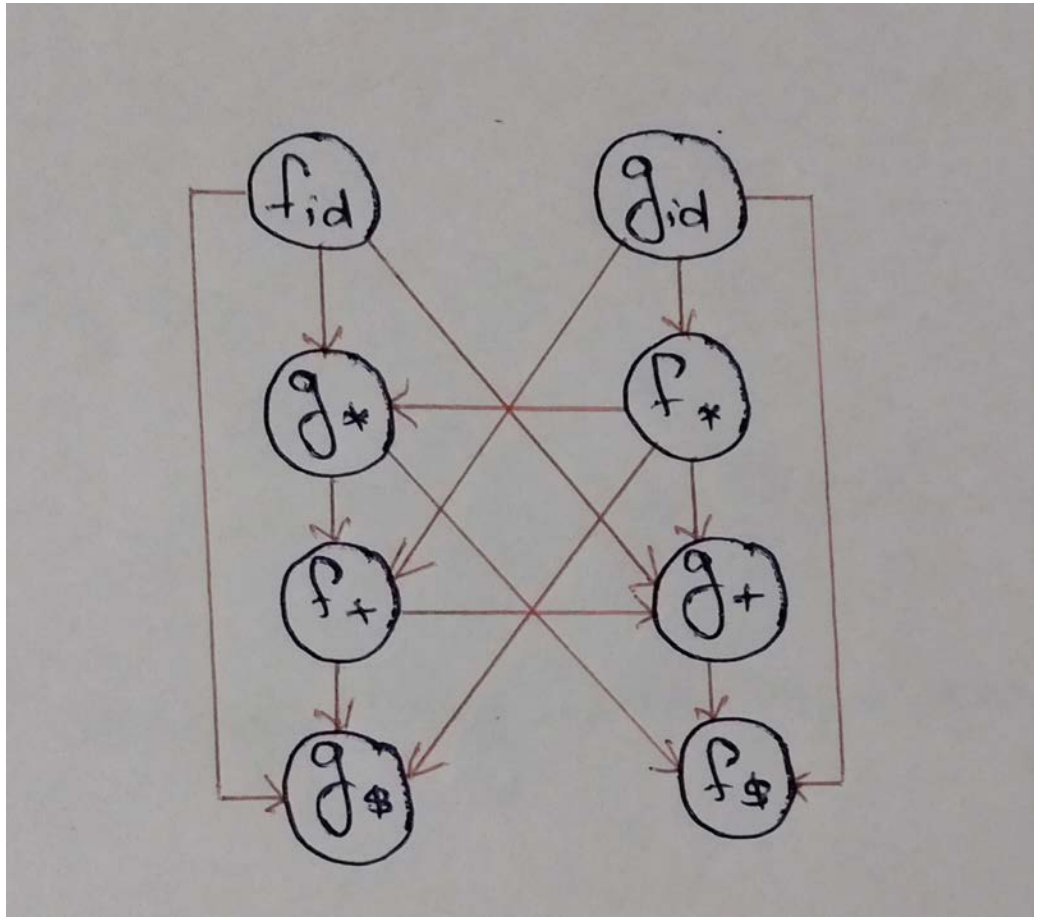
Operator function table

g

f

	id	+	*	$
id	-	$\dot{>}$	$\dot{>}$	$\dot{>}$
+	$\dot{<}$	$\dot{>}$	$\dot{<}$	$\dot{>}$
*	$\dot{<}$	$\dot{>}$	$\dot{>}$	$\dot{>}$
$	$\dot{<}$	$\dot{<}$	$\dot{<}$	-

Operator function table



Operator function table

$f_{id} \rightarrow g_{*} \rightarrow f_{+} \rightarrow g_{+} \rightarrow f_{\$}$

$g_{id} \rightarrow f_{*} \rightarrow g_{*} \rightarrow f_{+} \rightarrow g_{+} \rightarrow f_{\$}$

Operator Function Table

	id	+	*	$
f	4	2	4	0
g	5	1	3	0

Total no of entries 2n

Operator function table

$f_{+} > g_{+}$

$f_{+} < g_{*}$

<u>Disadvantages:</u>

- No blank entries in operator function table, hence error identification is less.

LR Parsing

- LR Parser|Compiler Design|Part# 20|Learn CSE Malayalam

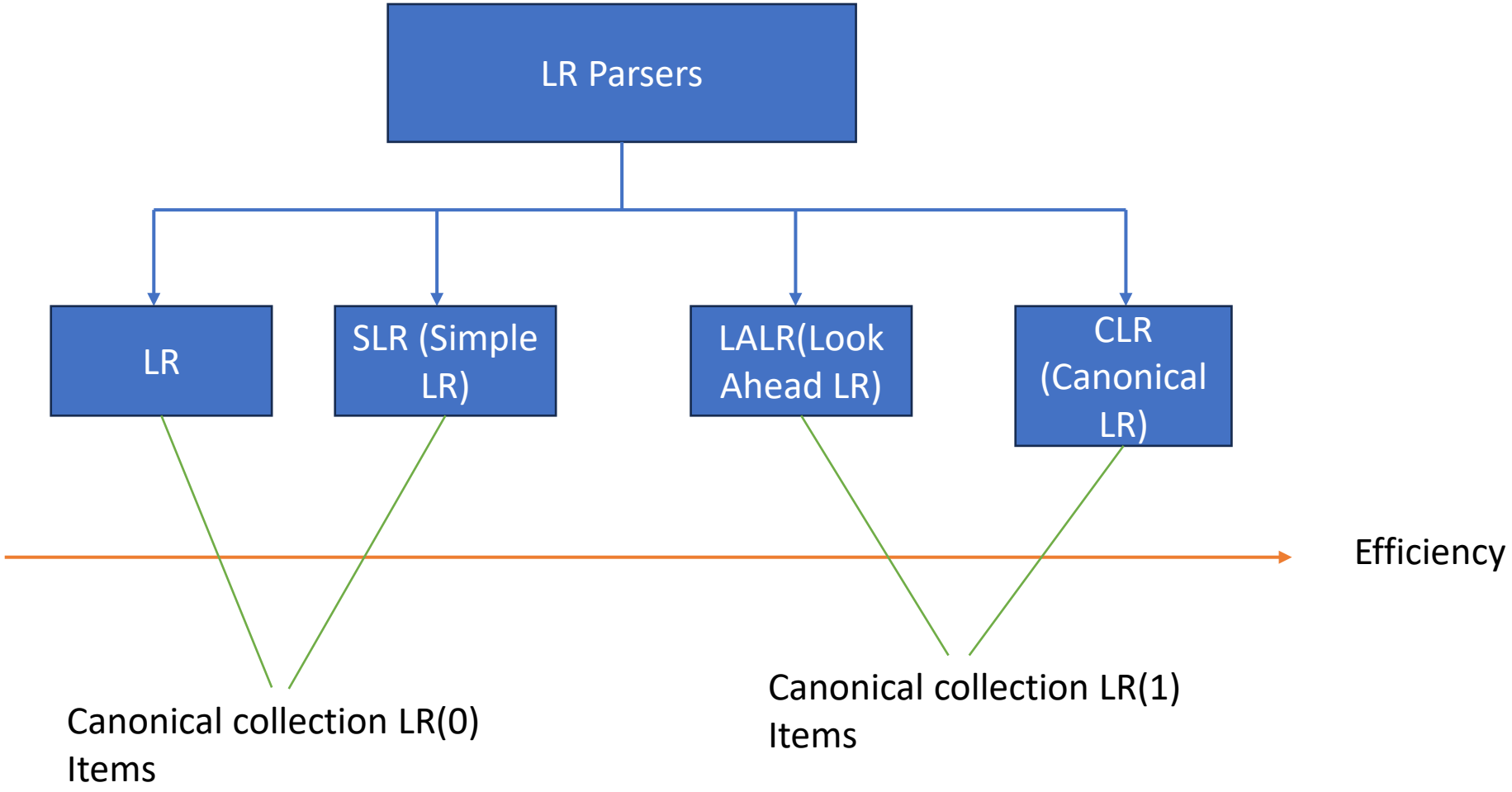
LR Parsing

- Most efficient method of bottom up parsing which can be used to parse the large class of context free grammars.
- LR(k):
  - L → Left to right scanning
  - R → Right most derivation in reverse order
  - k → no. of input symbols detected
- LR parsers can detect syntactic errors as soon as it is possible to do so on a left right scan of the input

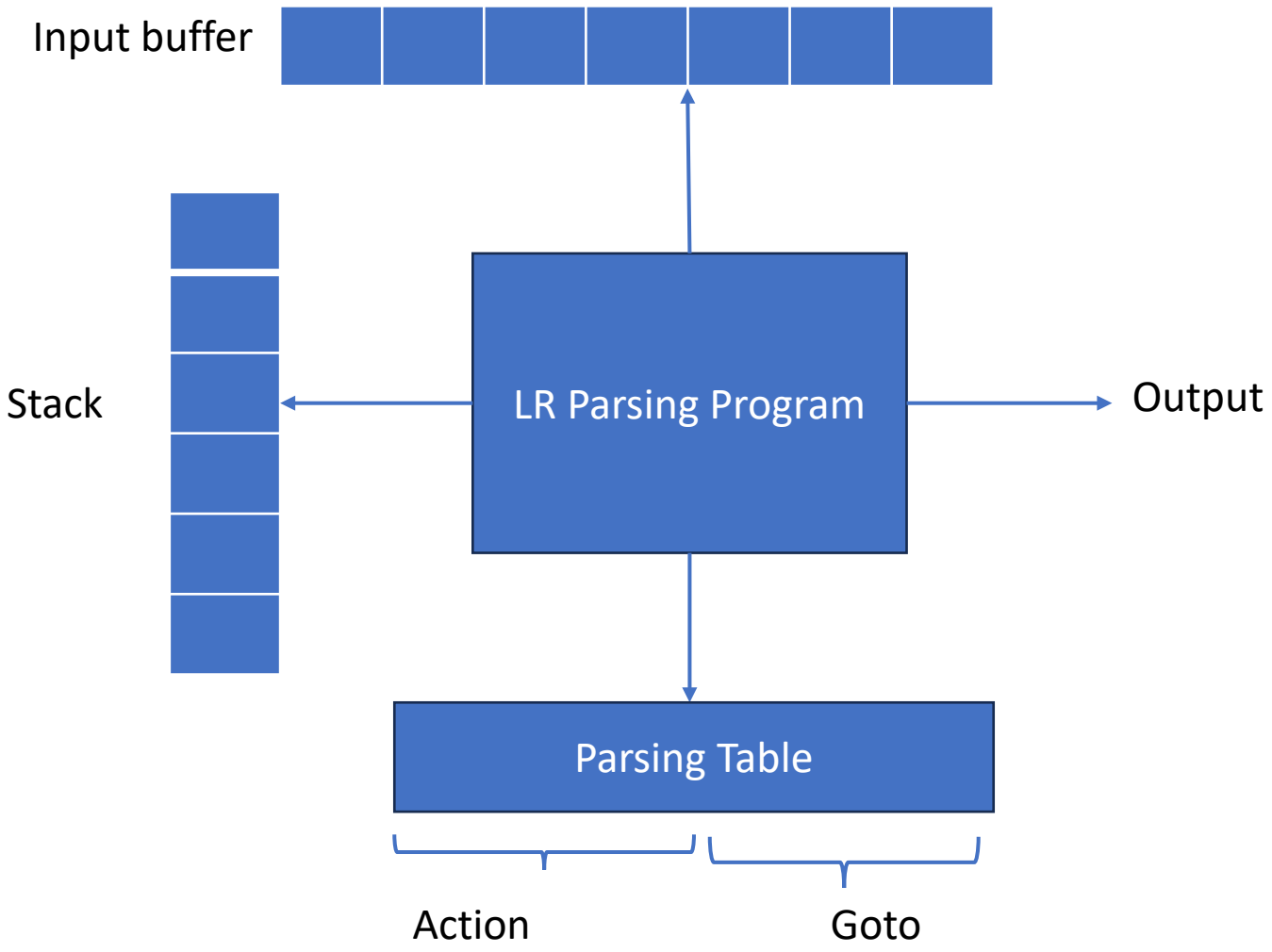
LR Parsing

- Most general non backtracking shift reduce parsing method
- The grammar is a proper subset of the class of grammars that can be parsed with predictive parsers.
- One of the drawback of the LR method is that it is too much work to construct an LR parsers by hand for a typical programming language grammar.
- A specialized tool, an LR parser generator is needed.
- Yacc – Yet Another Compiler Compiler

LR Parsing - Classification



LR Parsers - Structure



Terms used in LR parser

Items:

- An LR parser makes shift reduce decision by maintaining states, that represents set of items, to keep track of where we are in a parse

Terms used in LR parser

LR(0) items:

- An LR(0) item of a grammar G is a production of G with a dot at some position of the body
- Thus, production A → XYZ yields four items
  - A → .XYZ
  - A → X.YZ
  - A → XY.Z
  - A → XYZ.

Terms used in LR parsers

Canonical LR(0) Collection:

- One collection of sets of LR(0) items called the canonical LR(0) collection, provides the basic for constructing a deterministic finite automaton that is used to make parsing decision.
- To construct the canonical LR(0) collection for a grammar, an augmented grammar and two functions CLOSURE and GOTO are required

Terms used in LR parser

Augmented grammar:

- If G is a grammar with start symbol S, then G’, the augmented grammar for G, is G with a new start symbol S’ and the production S’→ S

Terms used in LR parser

Closure:

- If I is a set of items for a grammar G, then closure(I) is
  - Initially, add every item in I to Closure(I)
  - If A →α.Bβ is in Closure(I) and B → γ is a production, then add the item B → γ to Closure(I). Apply this rule until no more new items can be added to Closure(I)
    - Kernel items:- the initial item, S' → .S, and all items whose dots are not at the left end.
    - Non kernel items:- all items with their dots at the left end, except for S' → .S

Terms used in LR parser

Goto:

- The function is GOTO(I, X) where I is a set of items and X is a grammar symbol.
- GOTO(I, X) is defined to be the closure of the set of all items $[A \rightarrow \alpha X.\beta]$ such that $[A \rightarrow \alpha.X\beta]$ is in I.

Terms used in LR parser

LR(1) items:

- LR(1) items is a collection of LR(0) items and a lookahead symbol. For the augmented grammar the lookahead is $ and for the rest of the grammar with production A→α, the lookahead is FOLLOW(A).

- A → . ε
- A → ε.
- A → .

Canonical Collection of LR(0) Items

- Canonical Collection of LR(0) items|Compiler Design|Part# 21|Learn CSE Malayalam

Canonical Collection of LR(0) Items

Eg: $S \rightarrow AA$

$A \rightarrow aA \mid b$

Sol:

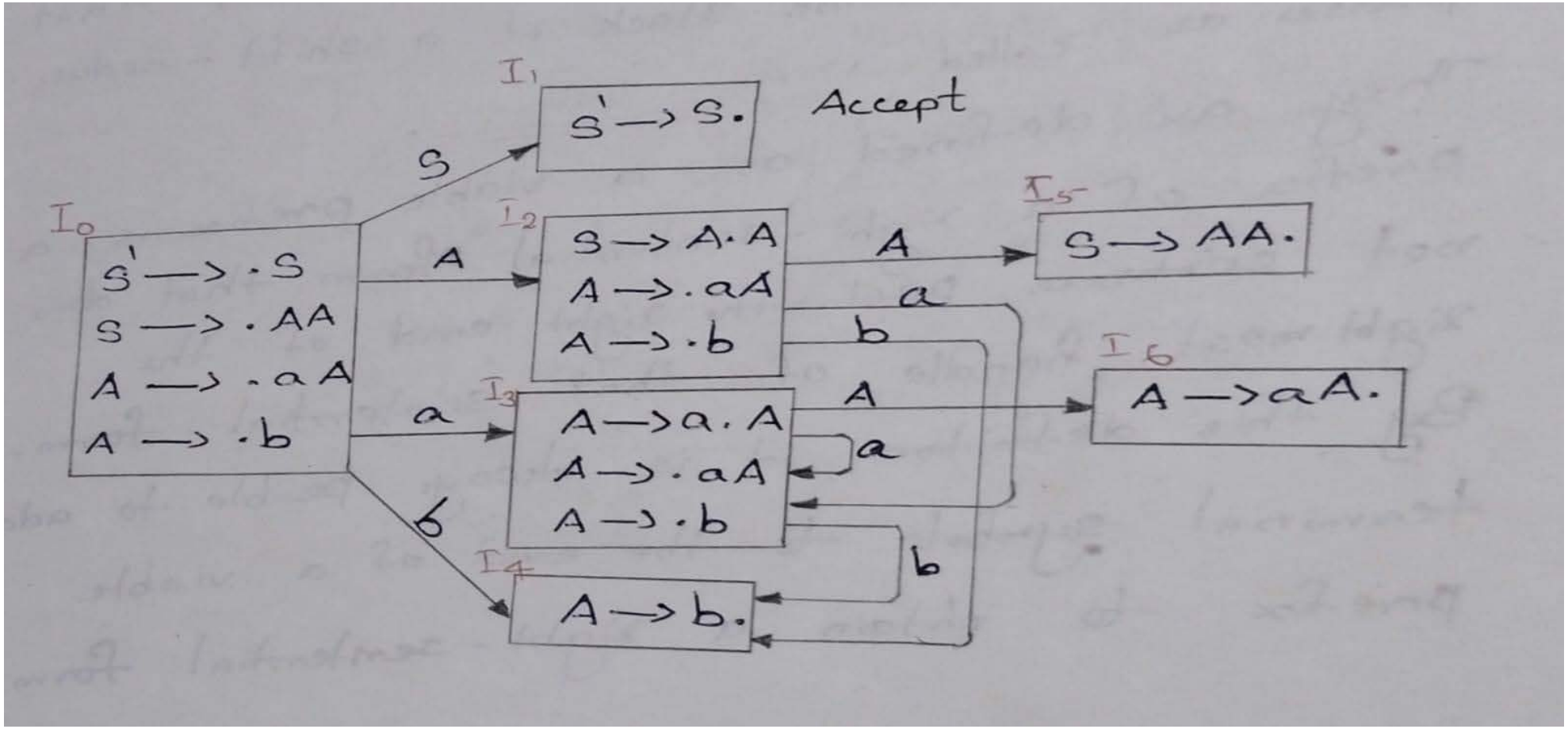
- First write the augmented grammar

$S' \rightarrow S$

$S \rightarrow AA$

$A \rightarrow aA \mid b$

Canonical Collection of LR(0) Items



Right sentential form

Why an LR(0) automata be used to make shift reduce decision?

- The LR(0) automaton for a grammar characterizes the strings of grammar symbols that can be appear on the stack of a shift reduce parser for the grammar.
- The stack contents must be a prefix of a right sentential forms.
- If the stack holds α and the rest of the input is x, then a sequence of reductions will take αx to S.
- In terms of derivation $S \Rightarrow^*_{rm} \alpha x$

Viable Prefixes

- The prefixes of right sentential forms that can appear on the stack of a shift reduce parser are called viable prefixes.
- They are defined as “a viable prefix of a right sentential form that does not extend beyond the rightmost handle of that sentential form”.
- By this definition, it is always possible to add terminal symbols to the end of a viable prefix to obtain a right sentential form.

Canonical collection of LR(1) Items

- Canonical Collection of LR(1) items|Compiler Design|Part# 22|Learn CSE Malayalam

Canonical collection of LR(1) Items

LR(1) items = LR(0) Items + a lookahead

Eg: $S \rightarrow AA$

$A \rightarrow aA \mid b$

Sol:

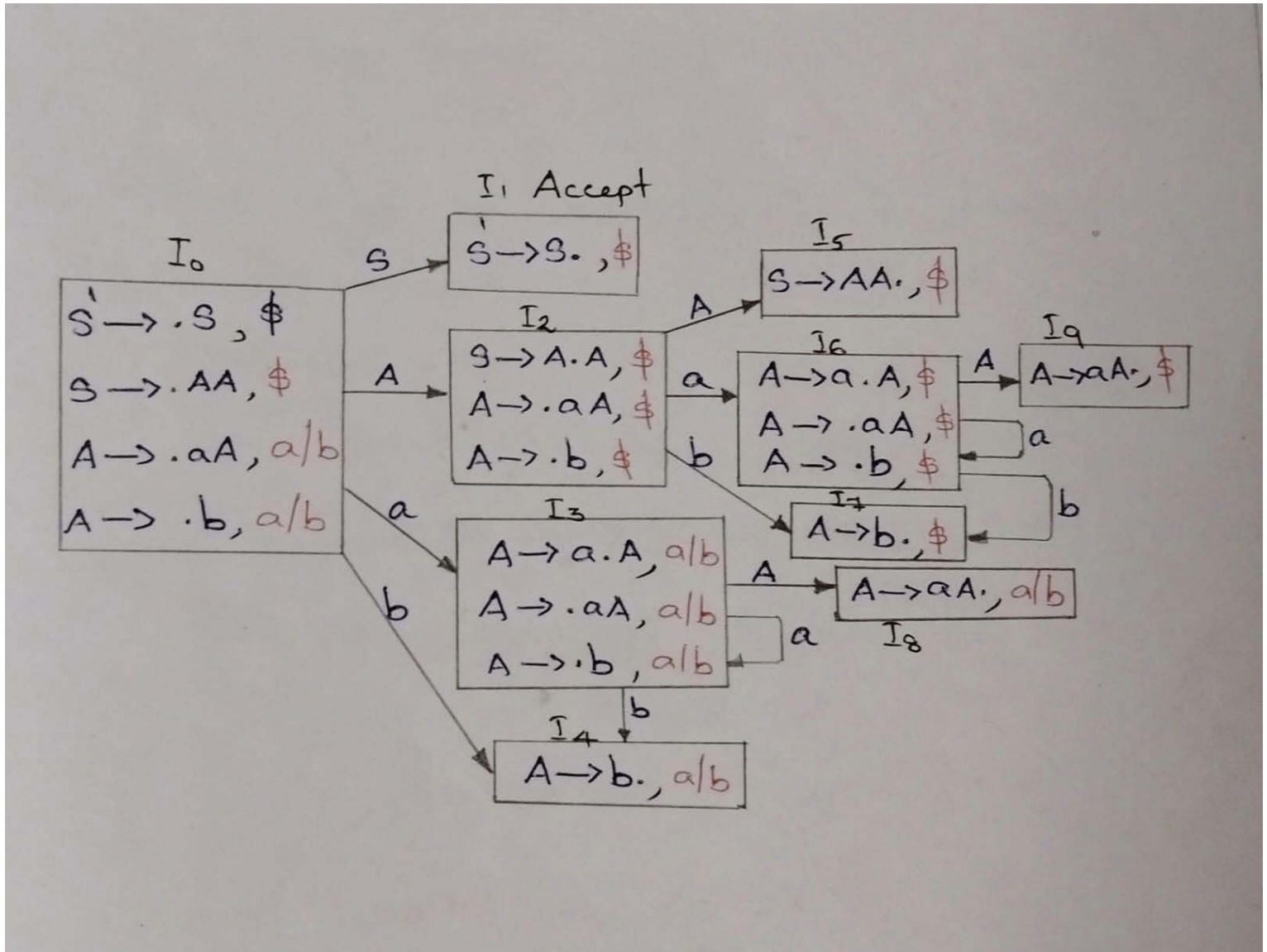
First write the augmented grammar

$S' \rightarrow S$

$S \rightarrow AA$

$A \rightarrow aA \mid b$

Canonical collection of LR(1) Items



LR(0) Parsing Table

- LR(0) Parsing Table | Compiler Design | Part# 23 | Learn CSE Malayalam

LR(0) Parsing Table

- The parsing table is divided into two parts:
  1. ACTION
  2. GOTO
- The code for the action are:
  - S_i means shift with stack state i
  - r_j means reduce by production number j
  - acc means accept
  - blank means error

LR(0) Parsing Table

- The value of GOTO[S, a] for terminal ‘a’ is found in ACTION field connected with the shift action on input ‘a’ for state S.
- The GOTO field gives GOTO[S, A] for non terminal A.

LR(0) Parsing Table - Example

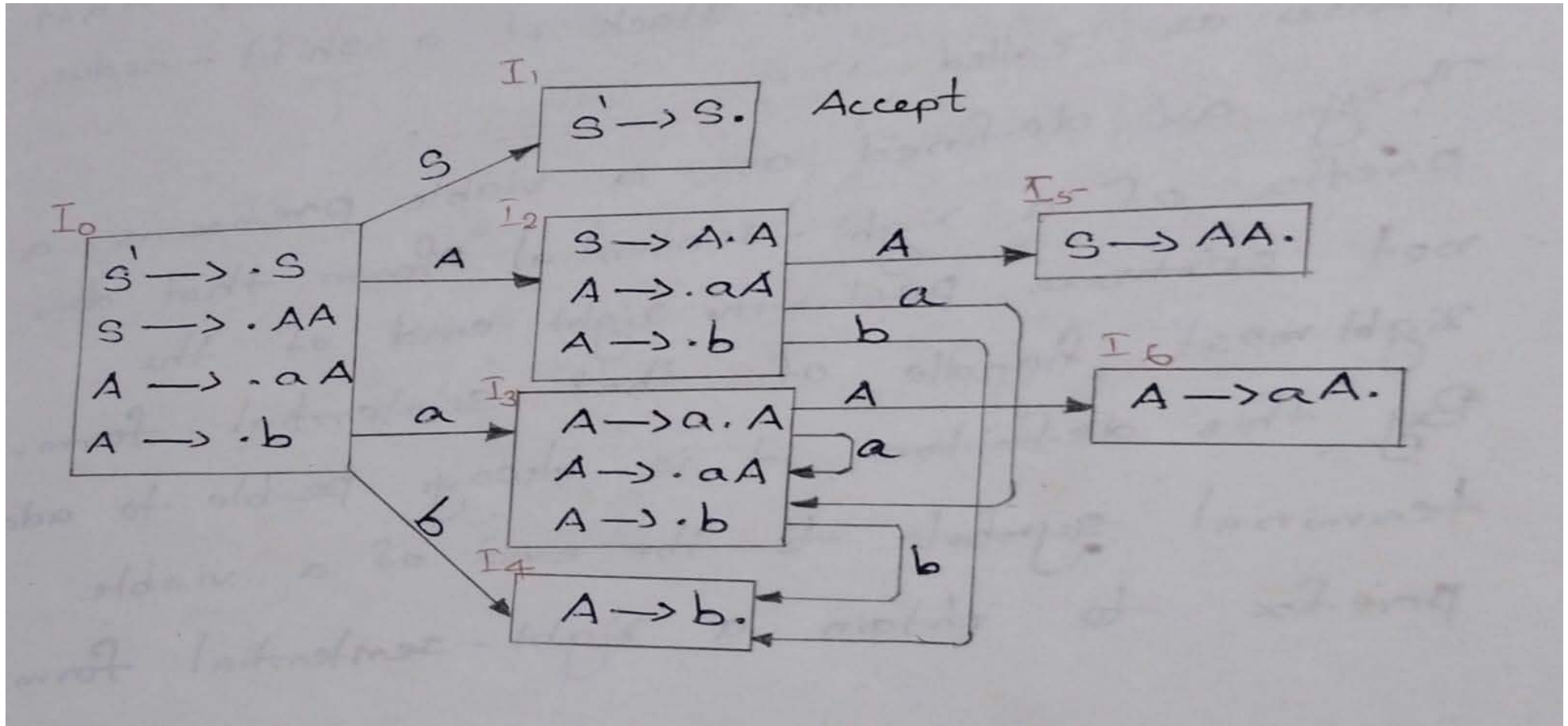
Eg:

Productions:

1) S → AA
2) A → aA
3) A → b

LR(0) Parsing Table - Example

- Consider the canonical collection of LR(0) items



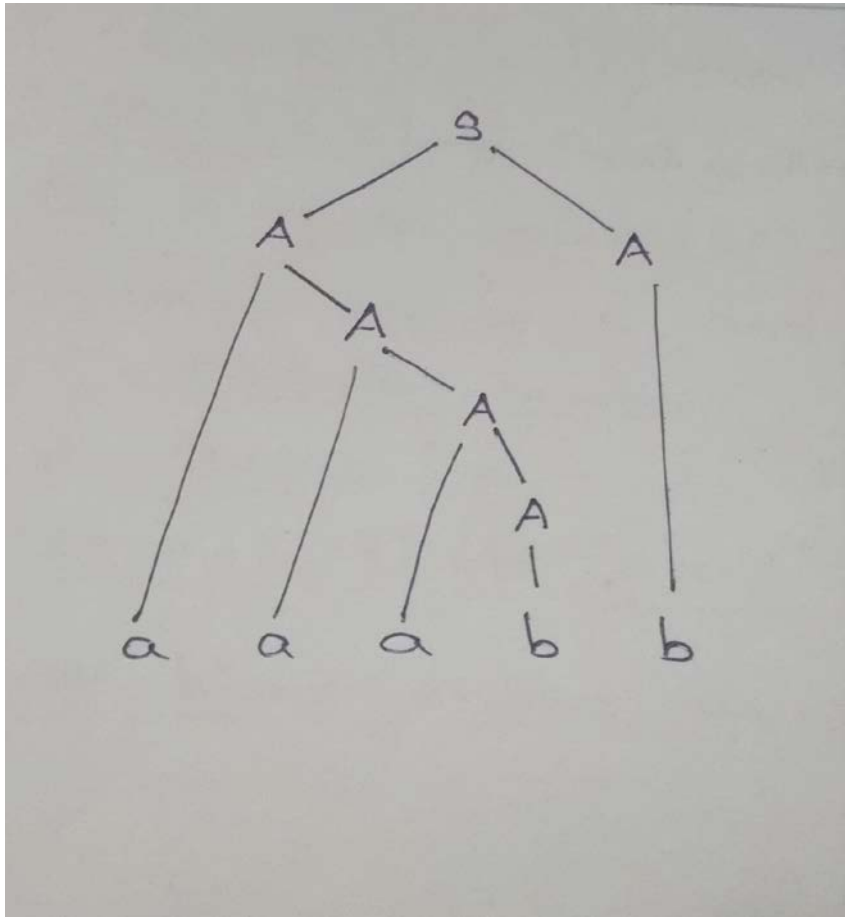
LR(0) Parsing Table - Example

STATE	ACTION			GOTO	
	a	**b**	**$**	**S**	**A**
0	S_3	S_4		1	2
1			acc		
2	S_3	S_4			5
3	S_3	S_4			6
4	r_3	r_3	r_3		
5	r_1	r_1	r_1		
6	r_2	r_2	r_2		

LR(0) Parsing Example with input string "aaabb"

STACK	INPUT	ACTION
0	aaabb$	Shift
0a3	aabb$	Shift
0a3a3	abb$	Shift
0a3a3a3	bb$	Shift
0a3a3a3b4	b$	Reduce A → b
0a3a3a3A6	b$	Reduce A → aA
0a3a3A6	b$	Reduce A → aA
0a3A6	b$	Reduce A → aA
0A2	b$	Shift
0A2b4	$	Reduce A → b
0A2A5	$	Reduce S → AA
0	$	Accept

LR(0) Example Parse Tree



LR(0) Parser

- In LR(0) parser the number of input symbols looking at a time is set to 0 because, in LR(0) parser the parse tree is generated without considering the next symbol of any input string.
- In the given input “aaabb”, the first ‘b’ generated its parent ‘A’ without considering the second ‘b’.
- The first ‘b’ generate its parent ‘A’ whatever the next symbol to it.

SLR(1) parsing Table

- SLR(1) Parsing Table|Compiler Design|Part# 25|Learn CSE Malayalam

SLR(1) parsing Table

- The parsing table is divide into
  1. ACTION
  2. GOTO
- If $A \rightarrow \alpha.a\beta$ is in I_i and and GOTO[I_i , a], then set ACTION[i, a] to “shift j (S_j)”. Here ‘a’ must be a terminal.
- If GOTO[I_i, A] = I_j, then GOTO[i, A] = j. Here ‘A’ must be a non terminal.
- If $A \rightarrow \alpha.$ is in I_i , then set ACTION[i, a] to “reduce $A \rightarrow \alpha$ (r_j)” for all ‘a’ in FOLLOW(A). Here ‘A’ may not be S’ and j is the production number.

SLR(1) parsing Table

- If S’ → S is in I_i, then set ACTION[i, $] to “accept”(acc).
- All blank entries are “error”

- The number of blank entries in SLR(1) >= LR(0) parser. Hence, the SLR(1) parser can identify error as soon as possible.

SLR(1) parsing Table - Example

- Production number

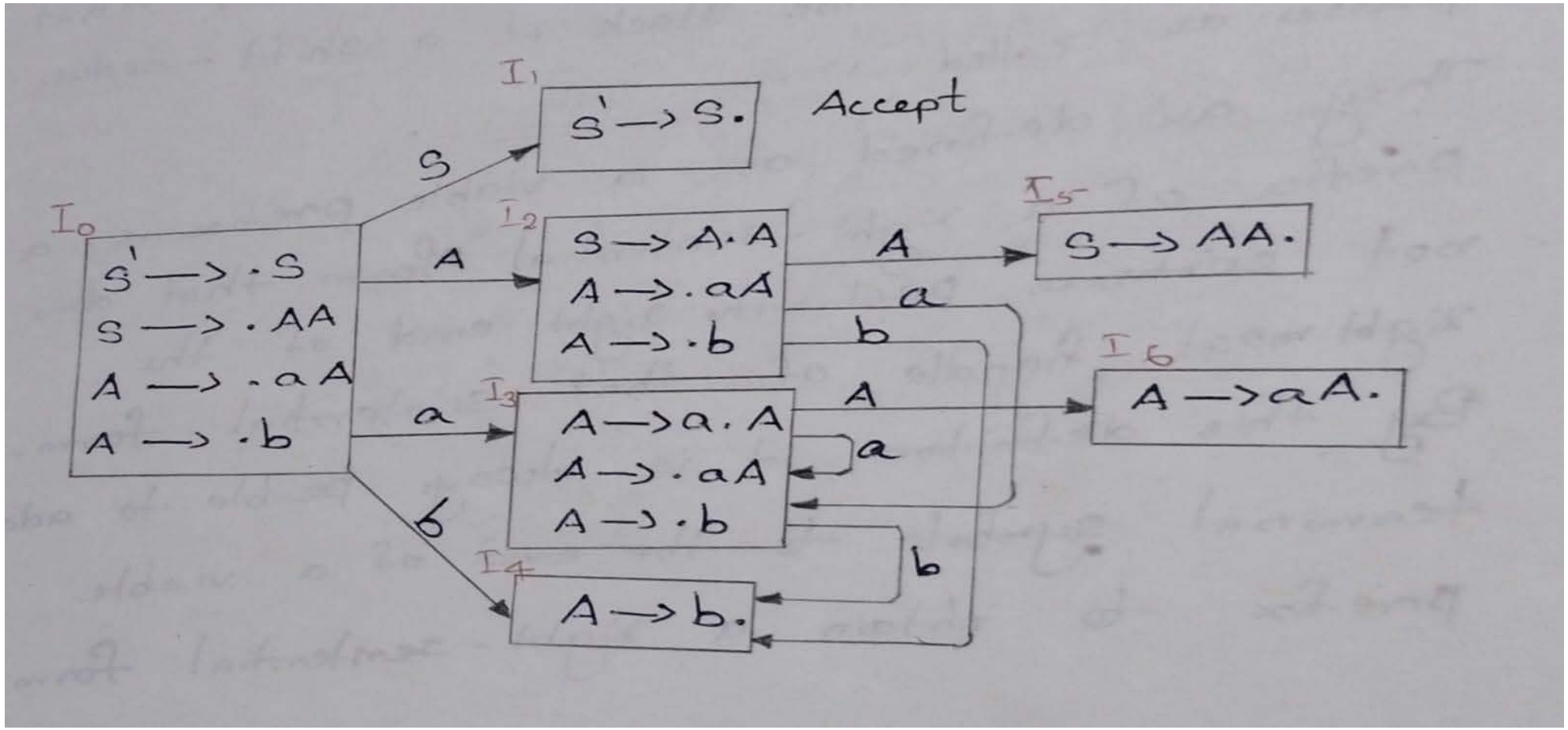
1) S → AA
2) A → aA
3) A → b

FOLLOW(S) = $

FOLLOW(A) = {a, b, $}

SLR(1) parsing Table - Example

- Consider the canonical collection of LR(0) items



SLR(1) parsing Table - Example

STATE	ACTION			GOTO	
	a	**b**	**$**	**S**	**A**
0	S_3	S_4		1	2
1			acc		
2	S_3	S_4			5
3	S_3	S_4			6
4	r_3	r_3	r_3		
5			r_1		
6	r_2	r_2	r_2		

CLR(1) Parsing Table

- CLR(1) Parsing Table|Compiler Design|Part# 26|Learn CSE Malayalam

CLR(1) Parsing Table

- The parsing table is constructed with the help of canonical collection of LR(1) items.
- The parsing table is divide into
  1. ACTION
  2. GOTO
- If $A \rightarrow \alpha.a\beta$ is in I_i and and GOTO[I_i , a], then set ACTION[i, a] to "shift j (S_j)". Here 'a' must be a terminal.
- If GOTO[I_i, A] = I_j, then GOTO[i, A] = j. Here 'A' must be a non terminal.

CLR(1) Parsing Table

- If $A \rightarrow \alpha.$ is in I_i , then set ACTION[i, a] to "reduce $A \rightarrow \alpha$ (r_j)" for all 'a' in FOLLOW(A). Here 'A' may not be S' and j is the production number.
- If S' → S is in I_i, then set ACTION[i, $] to "accept"(acc).
- All blank entries are "error"

- Number of rows of CLR(1) >= (LR(0) = SLR(1) = LALR(1))

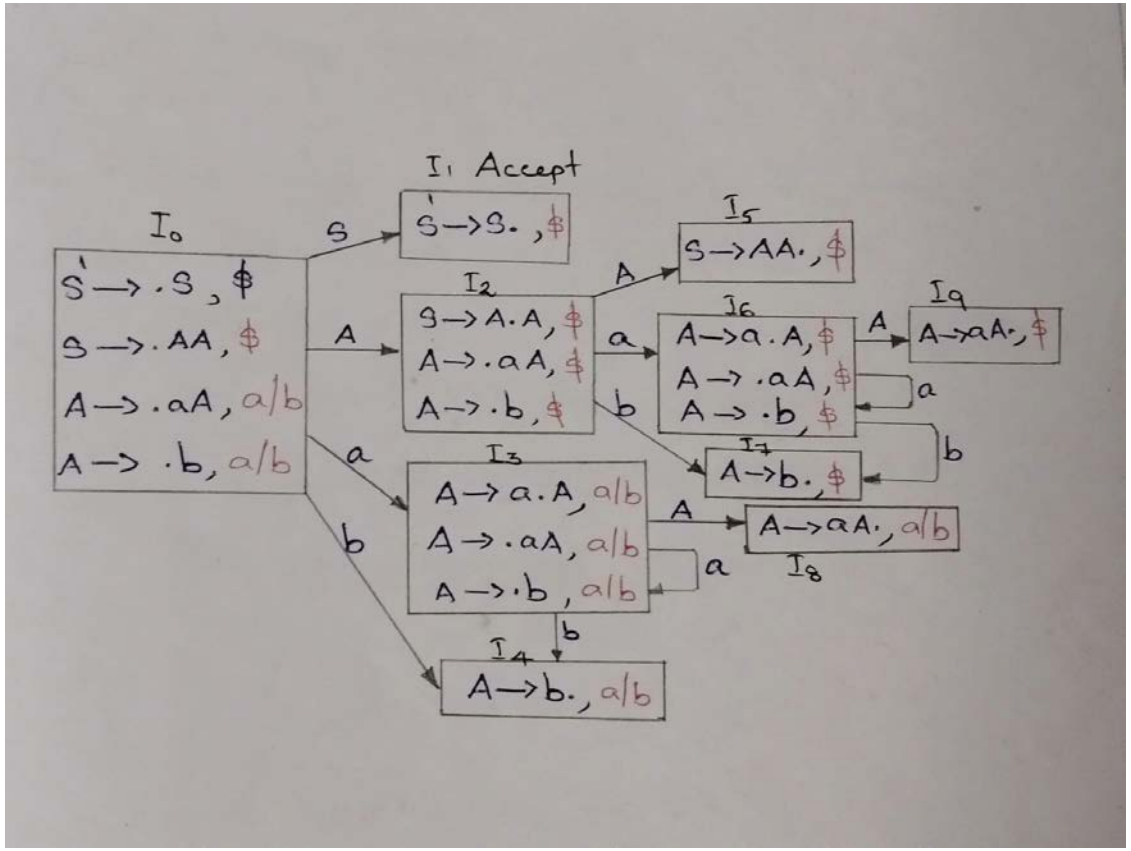
CLR(1) Parsing Table - Example

- Production number

1) S → AA
2) A → aA
3) A → b

CLR(1) Parsing Table - Example

- Consider the canonical collection of LR(1) items



CLR(1) Parsing Table - Example

STATE	ACTION			GOTO	
	a	**b**	**$**	**S**	**A**
0	S_3	S_4		1	2
1			acc		
2	S_6	S_7			5
3	S_3	S_4			8
4	r_3	r_3			
5			r_1		
6	S_6	S_7			9
7			r_3		
8	r_2	r_2			
9			r_2		

LALR(1) Parsing Table

- LALR(1) Parsing Table|Compiler Design|Part# 27|Learn CSE Malayalam

LALR(1) Parsing Table

- The parsing table is constructed with the help of canonical collection of LR(1) items.
- The parsing table is divide into
  1. ACTION
  2. GOTO
- For each core present among the set of LR(1) items, find all sets having that core and replace these sets by their union.

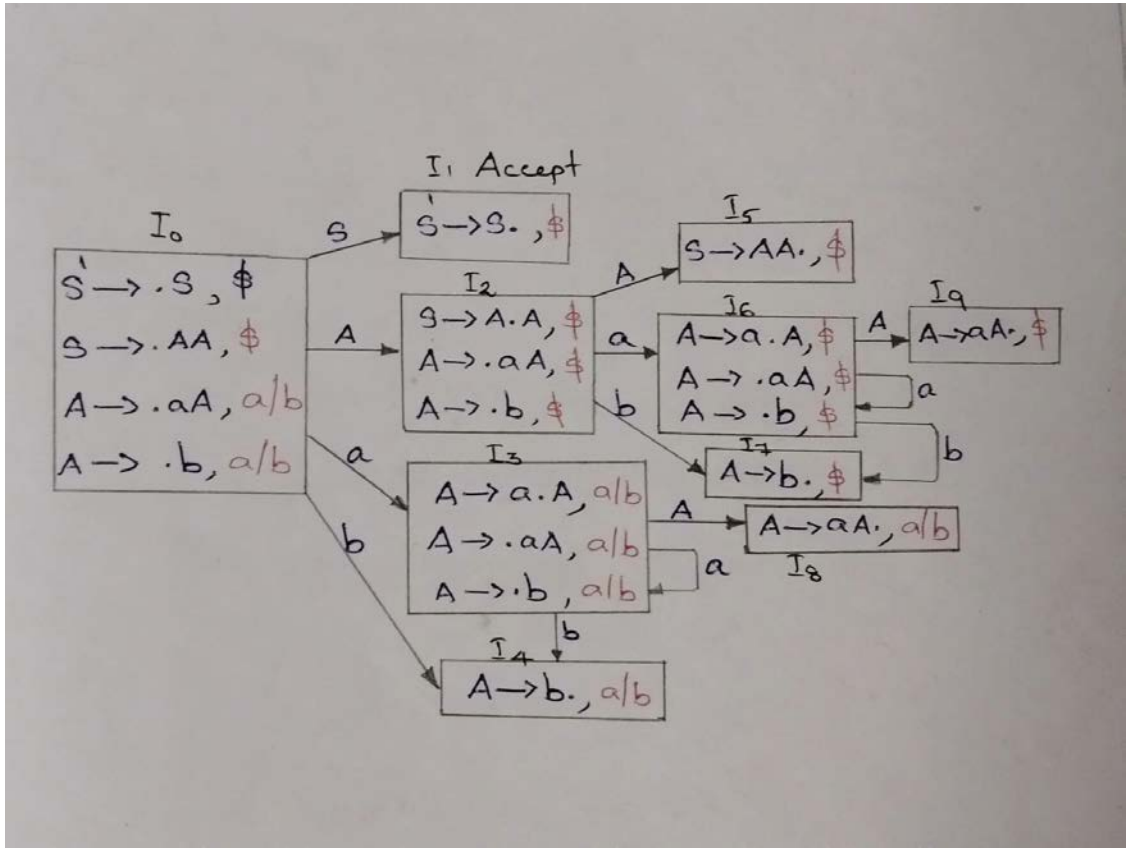
LALR(1) Parsing Table - Example

- Production number

1) S → AA
2) A → aA
3) A → b

LALR(1) Parsing Table - Example

- Consider the canonical collection of LR(1) items



LALR(1) Parsing Table - Example

- Here in the canonical collection, the states I_3 and I_6 are having same LR(0) items. Hence these two states can be combined in the parsing table and can writ eit as I_{36}.
- I_{36} = A → a.A, a|b|$

  A → .aA, a|b|$

  A → .b, a|b |$
- Similarly I_{47} = A →b. , a|b|$
- And I_{89} = A → aA. A | b | $

LALR(1) Parsing Table - Example

STATE	ACTION			GOTO	
	a	**b**	**$**	**S**	**A**
0	S_{36}	S_{47}		1	2
1			acc		
2	S_{36}	S_{47}			5
36	S_{36}	S_{47}			89
47	r_3	r_3	r_3		
5			r_1		
89	r_2	r_2	r_2		

LALR(1) Parsing Table

- Since, the number of rows is less in LALR(1) compared to CLR(1), it can utilize space and is efficient. Because it combining the states.

SR and RR Conflicts

- SR and RR Conflict|Compiler Design|Part# 28|Learn CSE Malayalam

SR and RR Conflicts

- Consider a state in canonical collection with final item, if
  - There are both shift and reduce in the same state, then it is said to be shift-reduce(SR) conflict.
  - There are two reduce actions in the same state, then it is called reduce – reduce(RR) conflict.

SR Conflict - Example

- Construct SLR(1) parsing table for the grammar

S → L=R | R

L → *R | id

R → L

SR Conflict - Example

Sol:

The augmented grammar

S’ → S

S → L = R | R

L → *R | id

R → L

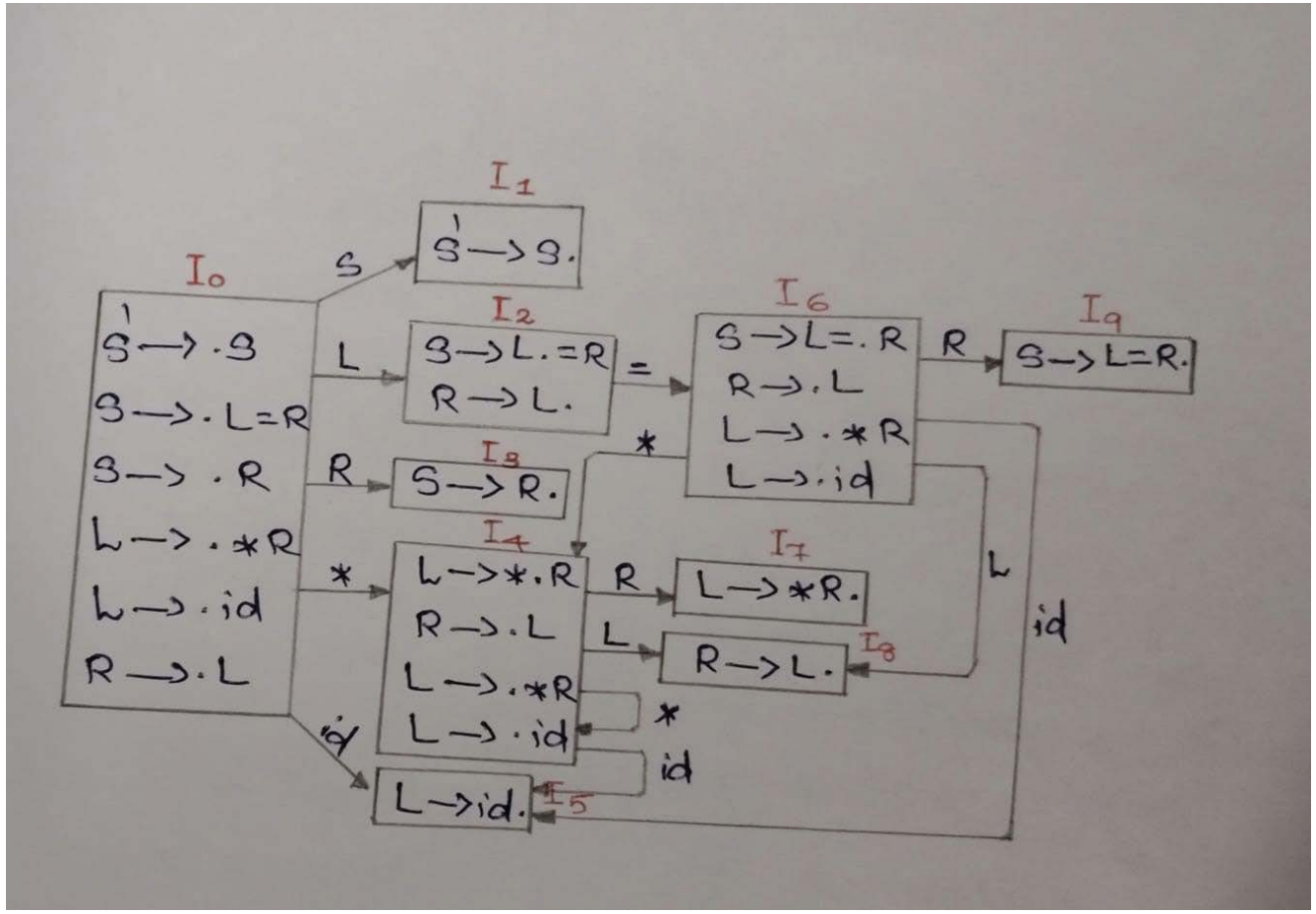
SR Conflict - Example

- Production numbers

1) S → L=R
2) S → R
3) L → *R
4) L → id
5) R → L

SR Conflict - Example

- Constructing the canonical collection of LR(0) items, we get



SR Conflict - Example

- SLR(0) Parsing Table

STATE	ACTION				GOTO		
	id	*	=	$	S	L	R
0	S_5	S_4			1	2	3
1				acc			
2			S_6/r_5	r_5			
. .	. .	. .	. .	. .	. .	. .	. .

- FOLLOW(R) = { $, =}

SR Conflict - Example

- S’ → S. is not a final item, it is an acceptance state. Hence, a shift action coming along with this LR item in a state is not considered to be a shift reduce conflict.
- Eg: a single state consists of S’ → S. and A → S.a is not SR conflict.

RR Conflict - Example

- Construct a SLR(1) parsing table for the grammar

S → AaAb | BbBa

A → a

B → a

RR Conflict - Example

- The augmented grammar is

S’ → S

S → AaAb | BbBa

A → a

B → a

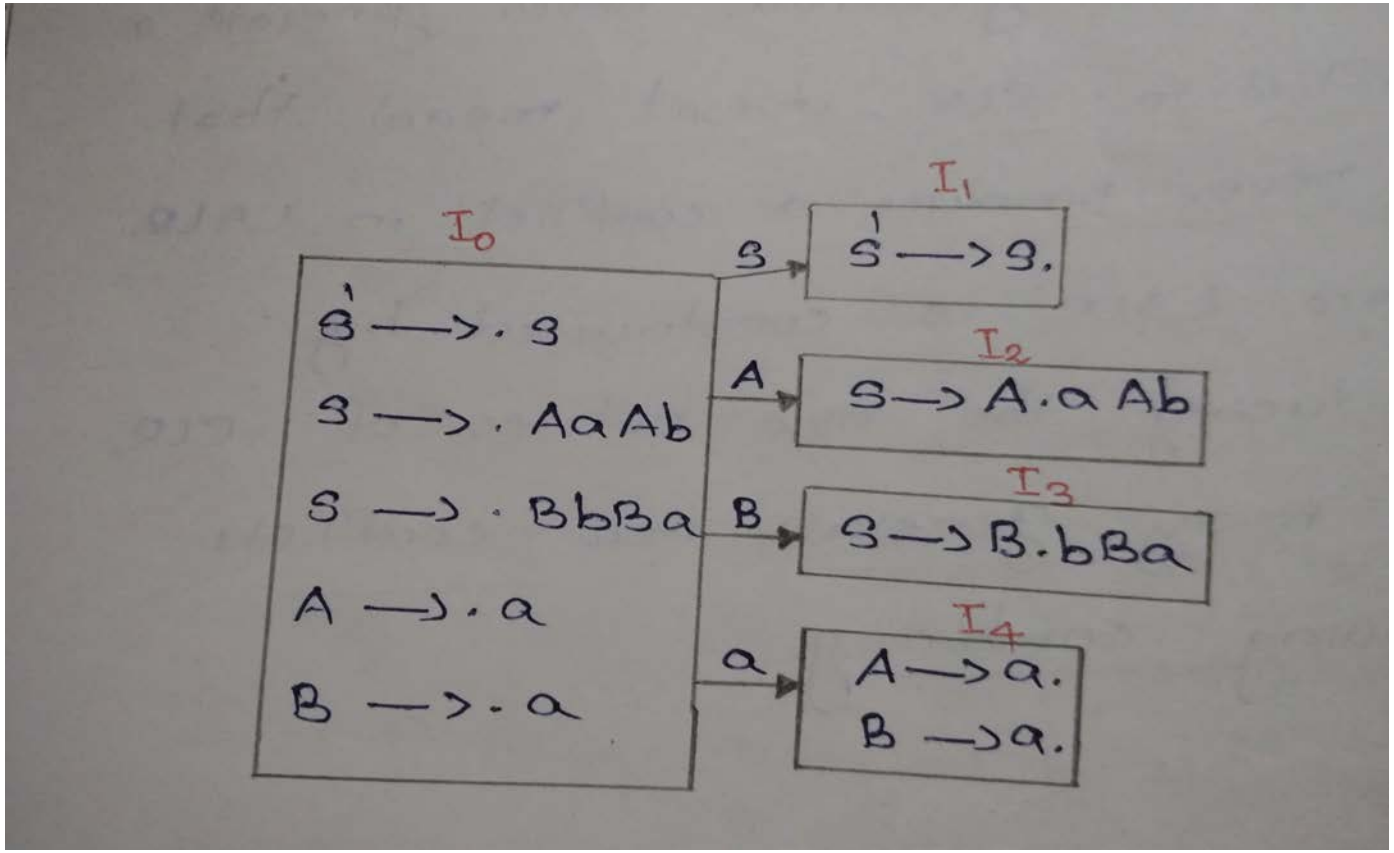
RR Conflict - Example

- The production numbers are as follows

1) S → AaAb
2) S → BbBa
3) A → a
4) B → a

RR Conflict - Example

- Here the canonical collection of LR(0) items are generated till a conflict



RR Conflict - Example

- SLR(1) parsing table

STATE	ACTION			GOTO		
	a	b	$	S	A	B
0	S_4			1	2	3
1			acc			
2						
3						
4	r_3/r_4	r_3/r_4				
. . .	. . .	. . .	. . .	. . .	. . .	. . .

SR and RR Conflict

- A conflict means parsing is impossible
- The LR(0) conflict can be reduced by SLR(1) because the reduce actions are only inserted in FOLLOW(A).
- The conflict in SLR(1)m can be reduced by CLR(1) because the reduce actions are only inserted in the lookaheads.
- A conflict in CLR can’t be reduced by LALR, because the row entries are combined in LALR.

SR and RR Conflict

- If a grammar generate a conflict in CLR, it is definitely generate a conflict in LALR.
- But, if a grammar never generate a conflict in CLR, doesn’t meant that it never produce a conflict in LALR.
- Since, LALR is constructed by reducing the row entries of CLR, it may generate new conflicts during combining.

LEX and YACC – Out of Syllabus

- Yacc: The Parser Generator|Compiler Design|Part# 29|Learn CSE Malayalam
- Lex:The Lexical Analyzer Generator|Compiler Design|Part# 30|Learn CSE Malayalam